JUSTIFICACIÓN USO DE IA LAB 7

Funciones en binary_search_tree.py:

- (Funciones del Reto) Remove, floor, ceiling, select, rank

Descripción: Se utilizó IA para mejorar el análisis y acercamiento general sobre cómo implementar estas funciones que consideramos útiles para el Reto 3.

Código:

```
14  def remove_node(root, key):
15      if root is None:
16          return None
17
18      cmp = default_compare(key, root)
19
20      if cmp == 0:
21          if root["left"] is None:
22              return root["right"]
23          elif root["right"] is None:
24              return root["left"]
25          else:
26              min_node = get_min_node(root["right"])
27              root["key"] = min_node["key"]
28              root["value"] = min_node["value"]
29              root["right"] = remove_node(root["right"], min_node["key"])
30      elif cmp == 1:
31          root["right"] = remove_node(root["right"], key)
32      else:
33          root["left"] = remove_node(root["left"], key)
34
35      return root
36
37  def remove(my_bst, key):
38      my_bst["root"] = remove_node(my_bst["root"], key)
39      return my_bst
40
41  def floor_node(root, key):
42      if root is None:
43          return None
44
45      cmp = default_compare(key, root)
46
47      if cmp == 0:
48          return root
49      elif cmp == -1:
```

```
        return floor_node(root["left"], key)
    else:
        right_floor = floor_node(root["right"], key)
        if right_floor is not None:
            return right_floor
        else:
            return root

def floor(my_bst, key):
    return floor_node(my_bst["root"], key)

def ceiling_node(root, key):
    if root is None:
        return None

    cmp = default_compare(key, root)

    if cmp == 0:
        return root
    elif cmp == 1:
        return ceiling_node(root["right"], key)
    else:
        left_ceiling = ceiling_node(root["left"], key)
        if left_ceiling is not None:
            return left_ceiling
        else:
            return root

def ceiling(my_bst, key):
    return ceiling_node(my_bst["root"], key)

def select_node(root, k):
    if root is None:
        return None

    left_size = size_tree(root["left"])
```

```
85      left_size = size_tree(root["left"])
86
87      if k < left_size:
88          return select_node(root["left"], k)
89      elif k > left_size:
90          return select_node(root["right"], k - left_size - 1)
91      else:
92          return root
93
94  def select(my_bst, k):
95      return select_node(my_bst["root"], k)
96
97  def rank_node(root, key):
98      if root is None:
99          return 0
100
101     cmp = default_compare(key, root)
102
103     if cmp == -1:
104         return rank_node(root["left"], key)
105     elif cmp == 1:
106         left_size = size_tree(root["left"])
107         return left_size + 1 + rank_node(root["right"], key)
108     else:
109         return size_tree(root["left"])
110
111 def rank(my_bst, key):
112     return rank_node(my_bst["root"], key)
113
```

Justificación: Para entender mejor la forma de implementar estas funciones, la lógica y el código que tienen detrás (porque no teníamos ni idea) y entender mejor su uso para hacer más eficiente el reto en un futuro.

Evidencia:

eben implementa
as). Para la entre
g(), select(), ran
el reto.

me interesa implementar estas funciones en bst para el reto del curso pero no se como aproximarme a cómo exactamente toca hacerlo, me podrias explicar el proceso de entendimiento de cada funcion y ademas unas pistas para poder hacer el codigo por mi cuenta?

¡Claro que sí! Implementar estructuras de datos recursivas como un árbol binario de búsqueda (BST) al principio puede imponer, pero una vez que entiendes la lógica detrás de cada función y el patrón de las **máscaras**, todo se vuelve mucho más natural.

Aquí te explico cómo entender el enfoque general (la relación entre la máscara y la función recursiva) y te doy pistas clave para que puedas escribir el código por tu cuenta.

¿Qué es una "máscara" y por qué se usa?

En estructuras recursivas, los métodos públicos que expone el BST (como `get(key)`) actúan como **máscaras** (o *wrappers*).

1. Función `floor(key)`

Entendimiento:

Busca la clave más grande dentro del árbol que sea **menor o igual** (`<=`) a la clave que le pasas como parámetro. Imagina que buscas un número en el árbol y no está; el `floor` sería el "piso", es decir, el elemento inmediatamente inferior más cercano.

Pistas para implementarla:

- **Caso base:** Si el nodo actual es `null`, retorna `null`.
- **Coincidencia exacta:** Si la clave del nodo actual es exactamente igual a la `key`, ¡ya la encontraste! Retorna esa clave.
- **Si la clave buscada es menor:** Si `key < nodo.key`, significa que el `floor` no puede estar en este nodo ni a su derecha (porque todo a la derecha es mayor). Tienes que buscar obligatoriamente en el **subárbol izquierdo**.
- **Si la clave buscada es mayor:** Si `key > nodo.key`, el nodo actual es un candidato válido para ser el `floor`. Sin embargo, vale la pena revisar el **subárbol derecho** por si hay una clave más grande que también sea menor o igual a `key`. Si la llamada derecha te devuelve algo válido, úsalo; si devuelve `null`, quédate con el nodo actual.

2. Función `ceiling(key)`

Entendimiento:

Es el opuesto simétrico del `floor`. Busca la clave más pequeña dentro del árbol que sea **mayor o igual** (`>=`) a la clave dada (el "techo" o siguiente valor hacia arriba).

Pistas para implementarla:

- **Caso base:** Si el nodo actual es `null`, retorna `null`.
- **Coincidencia exacta:** Si la clave del nodo actual es igual a la `key`, retorna esa clave.
- **Si la clave buscada es mayor:** Si `key > nodo.key`, el `ceiling` debe estar en el **subárbol derecho**, ya que el nodo actual y su izquierda son muy pequeños.
- **Si la clave buscada es menor:** Si `key < nodo.key`, el nodo actual es un candidato a `ceiling`. Pero revisa primero el **subárbol izquierdo** por si hay una clave más pequeña que aún cumpla con ser mayor o igual a `key`. Si la izquierda encuentra algo, retorna eso; de lo contrario, el nodo actual es tu respuesta.

3. Función `select(k)`

Entendimiento:

3. Función `select(k)`

Entendimiento:

Te permite encontrar la clave que ocupa la **posición** `k` (empezando desde el 0) si ordenaras todas las claves del árbol de menor a mayor (recorrido en orden). Por ejemplo, `select(0)` te da la clave más pequeña de todo el árbol. *Nota: Para esta función necesitas obligatoriamente que cada nodo guarde el tamaño de su subárbol (* `size` *).*

Pistas para implementarla:

- **El poder del tamaño izquierdo:** Cuenta cuántos elementos hay a la izquierda del nodo actual usando `int t = size(nodo.left);`.
- **Comparación con** `k`**:**
  - **Si** `t > k`**:** Significa que hay más elementos a la izquierda que la posición que buscas; por lo tanto, el elemento está en el **subárbol izquierdo**. Llama recursivamente a `select(nodo.left, k)`.
  - **Si** `t < k`**:** El elemento está en el **subárbol derecho**. Pero ojo: al pasar al lado derecho, debes descontar los elementos que ya dejaste atrás (el subárbol izquierdo más el nodo actual). Tu nuevo índice será `k - t - 1`.
  - **Si** `t == k`**:** ¡Bingo! El nodo actual es exactamente el elemento que ocupa la posición `k`. Retorna la clave del nodo actual

4. Función `rank(key)`

Entendimiento:

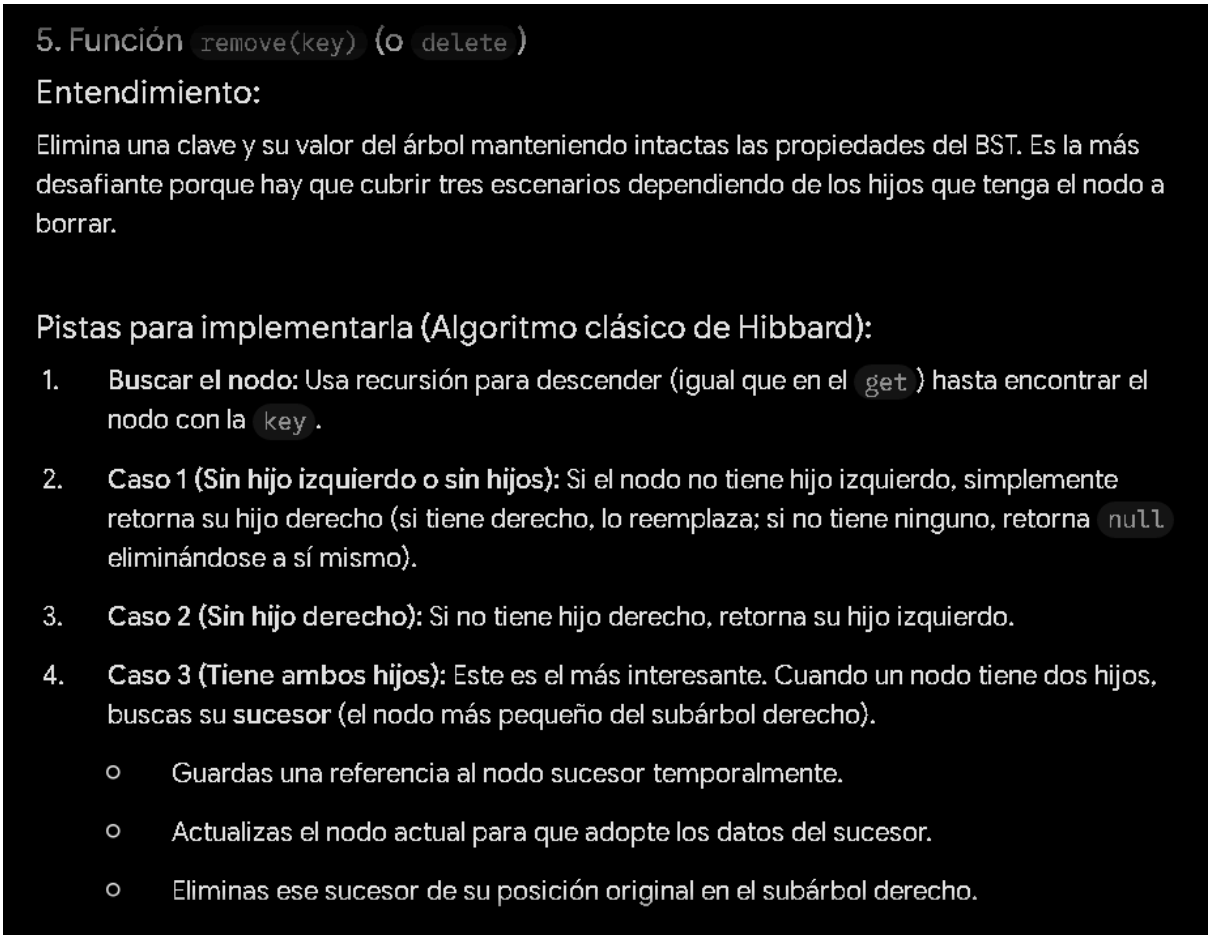
Es la operación inversa a `select`. Dada una clave, te dice **cuántas claves hay en el árbol que son estrictamente menores** a esa clave (es decir, su posición ordinal).

Pistas para implementarla:

- **Caso base:** Si el nodo actual es `null`, retorna `0`.
- **Si la clave buscada es menor:** Si `key < nodo.key`, significa que la clave debería estar a la izquierda, por lo que el ranking solo depende del **subárbol izquierdo.** Llama recursivamente a `rank(nodo.left, key)`.
- **Si la clave es mayor:** Si `key > nodo.key`, sabes que **todos** los elementos del subárbol izquierdo, más el nodo actual, son menores que tu clave. El resultado parcial es `1 + size(nodo.left)`, y a eso le sumas lo que encuentres recorriendo el **subárbol derecho** (`rank(nodo.right, key)`).
- **Si son iguales:** Si `key == nodo.key`, el número de elementos estrictamente menores es exactamente el tamaño de su subárbol izquierdo (`size(nodo.left)`).

5. Función `remove(key)` (o `delete`)

Entendimiento:



Conocimiento obtenido: Se logró entender con mayor claridad un conjunto de funciones muy útiles que nos pueden servir luego para realizar el reto, estas herramientas hacen que al momento de realizar los requerimientos se puedan hacer las cosas de forma más eficiente y con menos errores, además de que ayudan a crear y ejercitar conocimientos útiles a la hora de implementar funciones de bst.

Funciones en logic:

- New_logic, update_date_index y add_date_index

Descripción: Se utilizó la IA para un planteamiento general para antes de hacer el código para simplificar las tareas y evitar posibles errores.

Código:

```python
def new_logic():
    """ Inicializa el analizador

    Crea una lista vacia para guardar todos los crimenes
    Se crean indices (Maps) por los siguientes criterios:
    -Fechas

    Retorna el analizador inicializado.
    """
    analyzer = {'crimes': None,
                'dateIndex': None
                }

    analyzer['crimes'] = al.new_list()
    # TODO completar la creación del mapa ordenado
    analyzer['dateIndex'] = bst.new_map

    return analyzer
```

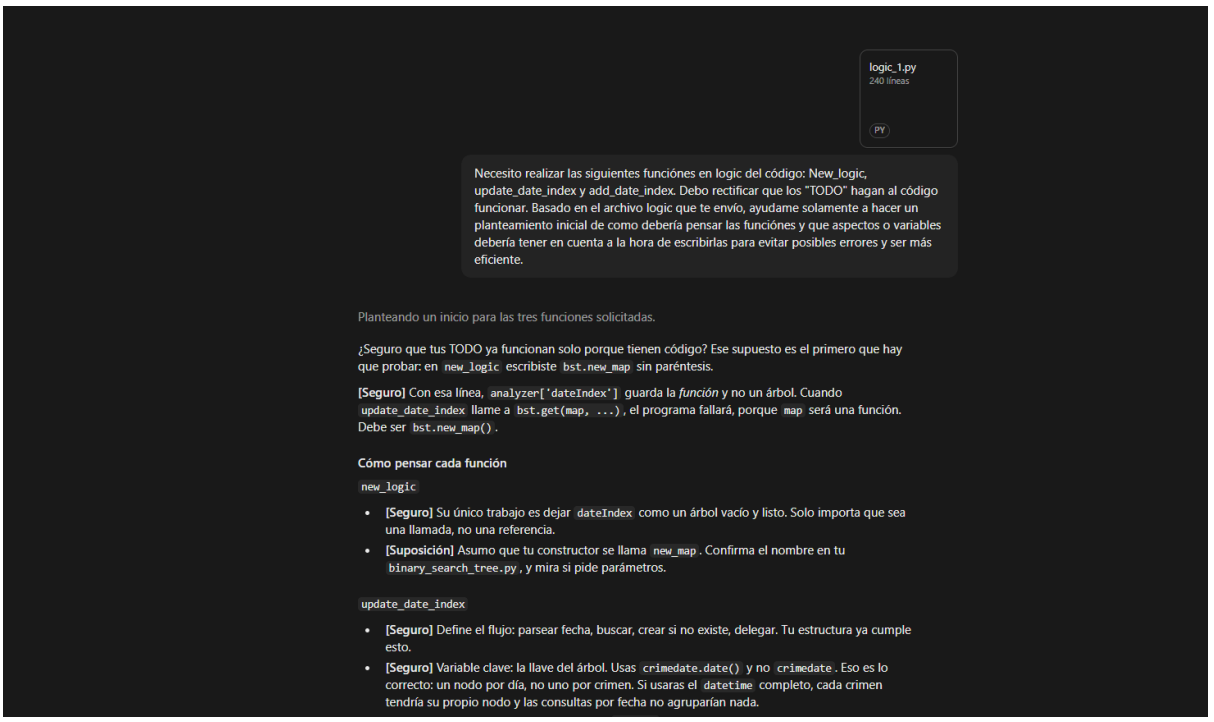
```python
def update_date_index(map, crime):
    """
    Se toma la fecha del crimen y se busca si ya existe en el arbol
    dicha fecha.  Si es asi, se adiciona a su lista de crimenes
    y se actualiza el indice de tipos de crimenes.

    Si no se encuentra creado un nodo para esa fecha en el arbol
    se crea y se actualiza el indice de tipos de crimenes
    """
    occurreddate = crime['OCCURRED_ON_DATE']
    crimedate = datetime.datetime.strptime(occurreddate, '%Y-%m-%d %H:%M:%S')
    entry = bst.get(map, crimedate.date())
    if entry is None:
        datentry = new_data_entry(crime)
        bst.put(map, crimedate.date(), datentry)
    else:
        datentry = entry
    add_date_index(datentry, crime)
    return map
```

```python
def add_date_index(datentry, crime):
    """
    Actualiza un indice de tipo de crimenes.  Este indice tiene una lista
    de crimenes y una tabla de hash cuya llave es el tipo de crimen y
    el valor es una lista con los crimenes de dicho tipo en la fecha que
    se está consultando (dada por el nodo del arbol)
    """
    lst = datentry['lstcrimes']
    al.add_last(lst, crime)
    offenseIndex = datentry['offenseIndex']
    offentry = lp.get(offenseIndex, crime['OFFENSE_CODE_GROUP'])
    if (offentry is None):
        # TODO Realice el caso en el que no se encuentre el tipo de crimen
        offentry = new_offense_entry(crime['OFFENSE_CODE_GROUP'], crime)
        al.add_last(offentry['lstoffenses'], crime)
        lp.put(offenseIndex, crime['OFFENSE_CODE_GROUP'], offentry)
    else:
        # TODO Realice el caso en el que se encuentre el tipo de crimen
        al.add_last(offentry['lstoffenses'], crime)
    return datentry
```

Justificación: Para evitar cometer errores que puedan comprometer el código, y entender mejor el detrás de cada función, se buscó con el prompt tener un primer vistazo al trabajo que había que hacer para asi agilizar y, como se mencionó antes, evitar posibles errores.

Evidencia:



Conocimiento obtenido: Se logró entender mejor cómo erán los índices de búsqueda y la lógica detrás.

Descripción: Se utilizó la IA para un planteamiento general para antes de hacer el código para simplificar las tareas y evitar posibles errores además para arreglar errores.

codigo

```
def get_crimes_by_range(analyzer, initialDate, finalDate):
    """
    Retorna el numero de crimenes en un rago de fechas.
    """
    # TODO Completar la función de consulta de crimenes por rango de fechas
    result_list = al.new_list()
    dates_list = al.new_list()

    bst.values_range(analyzer['dateIndex'], initialDate, finalDate, dates_list)

    for i in range(1, al.size(dates_list) + 1):
        datentry = al.get(dates_list, i)
        lst_crimes = datentry['lstcrimes']

        for j in range(1, al.size(lst_crimes) + 1):
            al.add_last(result_list, al.get(lst_crimes, j))

    return result_list
```

```python
def get_crimes_by_range_code(analyzer, initialDate, offensecode):
    """
    Para una fecha determinada, retorna el numero de crimenes
    de un tipo especifico.
    """
    # TODO Completar la función de consulta de crimenes por tipo de crimen en una fecha
    result_list = al.new_list()
    dates_list = al.new_list()

    datentry = bst.get(analyzer['dateIndex'], initialDate)

    for i in range(1, al.size(dates_list) + 1):
        datentry = al.get(dates_list, i)
        lst_crimes = datentry['lstcrimes']

        for j in range(1, al.size(lst_crimes) + 1):
            al.add_last(result_list, al.get(lst_crimes, j))

    return result_list
```

Justificación por desconocimiento se le pidió a la IA que brindara asesoramiento para la verificacion del codigo y que repara errores que pudiera haber cometido

evidencia

Quiero que revises el código lo corrijas

def get_crimes_by_range(analyzer, initialDate, finalDate):
"""

Hay algunos detalles importantes que corregir en ambos códigos, principalmente relacionados con **los límites de los ciclos (** `range` **)** en las estructuras de datos tipo lista (`array_list`), ya que en este tipo de implementaciones los índices suelen comenzar en **1** y van hasta el tamaño total inclusive (`+ 1`), y también hay un error de lógica en la segunda función (`get_crimes_by_range_code`) donde se intentaba iterar sobre una lista vacía (`dates_list`).

Aquí tienes el código corregido y optimizado:

1. `get_crimes_by_range`

Corrección realizada: En el ciclo `range(1, al.size(dates_list))` faltaba el `+ 1` , lo que provocaba que el último elemento de la lista de fechas no se evaluara. Lo mismo ocurría con los crímenes.

Python

```python
def get_crimes_by_range(analyzer, initialDate, finalDate):
    """
    Retorna el numero de crimenes en un rago de fechas.
    """
    result_list = al.new_list()
    dates_list = al.new_list()

    bst.values_range(analyzer['dateIndex'], initialDate, finalDate, dates_list)

    # Se agrega '+ 1' para incluir e imo elemento, ya que las listas empiezan
    for i in range(1, al.size(dates_list) + 1):
```

Que aprendí: un mejor manejo de rangos y estructuras de análisis.